

Практика 12. Трансформеры монад

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ **Compose**
- ▶ Монады, стандартные монады
- + Трансформеры монад

Содержание

1. Recall: monads
2. Love. Death. Transformers
3. Материалы

Содержание

1. Recall: monads
2. Love. Death. Transformers
3. Материалы

Абстрактный тип данных — Monad

△ АД — интерфейс конструирования обитателей некоторого множества

e.g. АД стек: операции `push(elem)` и `pop -> elem`

e.g. АД `Monad`: операции `pure` и `>>=`

e.g. Множества: `[]`, `Maybe`, `Either` e

e.g. Даже множества функций: `newtype State s a = State (s -> (a, s))`

todo Как следующую функцию сконструировать с помощью `Monad`?

```
computation :: s -> (a, s)
```

```
computation s = let s' = f s in (g s, s')
```

Абстрактный тип данных — Monad

△ АТД — интерфейс конструирования обитателей некоторого множества

e.g. АТД стек: операции `push(elem)` и `pop -> elem`

e.g. АТД **Monad**: операции `pure` и `>>=`

e.g. Множества: `[]`, `Maybe`, `Either e`

e.g. Даже множества функций: `newtype State s a = State (s -> (a, s))`

todo Как следующую функцию сконструировать с помощью **Monad**?

```
computation :: s -> (a, s)
```

```
computation s = let s' = f s in (g s, s')
```



```
computation = runState do
```

```
  s <- get      -- get      ::          s -> (s , s)
```

```
  modify f      -- modify :: (s -> s) -> (s -> ((), s))
```

```
  pure (g s)    -- pure    :: a ->      (s -> (a , s))
```

Вычисления из сложных множеств (доменов)

- ▶ Вычисление с двумя ячейками состояния лежит в $s \rightarrow s' \rightarrow ((a, s), s')$
- ▶ Для этого множества нужно определить все операции АД:
- ▶ Удобный способ конструирования обитателей: **Monad**
- ▶ Функции `get`, `put`, `modify` для s
- ▶ Те же функции `get2`, `put2`, `modify2` для $s' \dots$

NB Приходится переопределять всё каждый раз для новых доменов!

- ▶ Проблема известна как *expression problem*, *stable denotations*, *extensible interpreters*¹

¹<https://okmij.org/ftp/Computation/having-effect.html>.

Супер-монады (☁)

- ▶ Можно делать Super Monad сразу со всем необходимым

e.g. `newtype Super env err a = Super { runSuper :: env -> IO (Either err a) }`

e.g. `newtype Super effs a = Super { runSuper :: IO a }`

- Медленно работает, не zero-cost abstraction²
- Не модульно — сложно поддерживать и использовать частями
- Не получится легко подменять реализации и домены

²Zero-cost не значит бесплатно, а значит — если не используешь, то не платишь за её присутствие.

Монады не композируются (☹️)

► `join :: (Monad m, Monad n) => Compose m n (Compose m n a) -> Compose m n a`

► Запишем без обёрток для наглядности:

`join :: (Monad m, Monad n) => m (n (m (n a))) -> m (n a)`

► Можно попробовать переставить `m` и `n` местами и свести к двум `join`

NB Но монады в общем случае не коммутируют

e.g. Попробуем переставить `Reader` и `Writer`

1. `Reader e (Writer w a) ≅ e -> (w, a)`

2. `Writer w (Reader e a) ≅ (w, e -> a)`

* Алгебраически видно, что это не изоморфные типы: $(w * a)^e \neq w * a^e$

► Разумной реализации тоже не получается, перестановка от 1 к 2 теряет лог:

`dist :: Monoid w => (e -> (w, a)) -> (w, e -> a)`

`dist q = (mempty, \e -> fst (q e))`

Содержание

1. Recall: monads

2. Love. Death. Transformers

3. Материалы

Обобщаем до семейства множеств вычислений

- ▶ Программа с одной глобальной переменной задаёт функцию $s \rightarrow (a, s)$
- ▶ А если глобальных переменных две — $s \rightarrow s' \rightarrow ((a, s), s')$
- ▶ Оба варианта представимы как частные случаи семейства $s \rightarrow m(a, s)$

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Выберите подходящие m

todo Реализуйте `get` и `put` для `StateT`

todo Как сконструировать `s` с помощью `Monad (StateT s m)`?

```
computation :: s -> (a, s); computation s = let s' = f s in (g s, s')
```

Обобщаем до семейства множеств вычислений

- ▶ Программа с одной глобальной переменной задаёт функцию $s \rightarrow (a, s)$
- ▶ А если глобальных переменных две — $s \rightarrow s' \rightarrow ((a, s), s')$
- ▶ Оба варианта представимы как частные случаи семейства $s \rightarrow m(a, s)$

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Выберите подходящие m

todo Реализуйте `get` и `put` для `StateT`

todo Как сконструировать s с помощью `Monad (StateT s m)`?

```
computation :: s -> (a, s); computation s = let s' = f s in (g s, s')
```



Первый: $m\ a = \text{Identity}\ a$, второй: $m\ a = s' \rightarrow (a, s')$

Обобщаем до семейства множеств вычислений

- ▶ Программа с одной глобальной переменной задаёт функцию $s \rightarrow (a, s)$
- ▶ А если глобальных переменных две — $s \rightarrow s' \rightarrow ((a, s), s')$
- ▶ Оба варианта представимы как частные случаи семейства $s \rightarrow m (a, s)$

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Выберите подходящие m

todo Реализуйте `get` и `put` для `StateT`

todo Как сконструировать s с помощью `Monad (StateT s m)`?

```
computation :: s -> (a, s); computation s = let s' = f s in (g s, s')
```

⚡ Первый: $m\ a = \text{Identity}\ a$, второй: $m\ a = s' \rightarrow (a, s')$

⚡ `get :: Monad m => StateT s m s; get = StateT \s -> pure (s, s)`

Обобщаем до семейства множеств вычислений

- ▶ Программа с одной глобальной переменной задаёт функцию $s \rightarrow (a, s)$
- ▶ А если глобальных переменных две — $s \rightarrow s' \rightarrow ((a, s), s')$
- ▶ Оба варианта представимы как частные случаи семейства $s \rightarrow m(a, s)$

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Выберите подходящие m

todo Реализуйте `get` и `put` для `StateT`

todo Как сконструировать s с помощью `Monad (StateT s m)`?

```
computation :: s -> (a, s); computation s = let s' = f s in (g s, s')
```

⚡ Первый: $m\ a = \text{Identity}\ a$, второй: $m\ a = s' \rightarrow (a, s')$

⚡ `get :: Monad m => StateT s m s`; `get = StateT \s -> pure (s, s)`

⚡ `computation = runIdentity . runStateT do`

```
  s <- get    -- get      ::          s -> Identity (s , s)
```

```
  modify f    -- modify :: (s -> s) -> (s -> Identity ((), s))
```

```
  pure (g s)  -- pure    :: a ->          (s -> Identity (a , s))
```

Работа с вложенным состоянием

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Реализуйте способ поработать с вложенным состоянием

```
StateT s' Identity a -> StateT s (StateT s' Identity) a
(s' -> (a, s'))      -> (s -> s' -> ((a, s), s'))
```

todo Реализуйте для произвольной вложенной монады

Работа с вложенным состоянием

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Реализуйте способ поработать с вложенным состоянием

```
StateT s' Identity a -> StateT s (StateT s' Identity) a
(s' -> (a, s'))      -> (s -> s' -> ((a, s), s'))
```

todo Реализуйте для произвольной вложенной монады



```
liftState :: (s' -> (a, s')) -> (s -> s' -> ((a, s), s'))
liftState comp = \s -> \s' -> let (x, new) = comp s' in ((x, s), new)
```

Работа с вложенным состоянием

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

todo Реализуйте способ поработать с вложенным состоянием

```
StateT s' Identity a -> StateT s (StateT s' Identity) a
(s' -> (a, s'))      -> (s -> s' -> ((a, s), s'))
```

todo Реализуйте для произвольной вложенной монады

```
⚡ liftState :: (s' -> (a, s')) -> (s -> s' -> ((a, s), s'))
liftState comp = \s -> \s' -> let (x, new) = comp s' in ((x, s), new)

⚡ liftState :: Monad m => m a -> StateT s m a
liftState ma = StateT \s -> (,s) <$> ma
```

Lifting

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

- Перегрузим `lift` по трансформеру с помощью класса типов

```
class MonadTrans t where
    lift :: Monad m => m a -> t m a
```

`todo` Реализуйте `instance MonadTrans (StateT s)`

`todo` Сконструируйте в терминах трансформеров следующее вычисление

```
computation :: Int -> (Int, String)
computation = \e -> (e + 2, show e ++ show (e + 1))
```

Lifting

```
newtype StateT s m a = StateT { runStateT :: s -> m (a, s) }
```

- Перегрузим `lift` по трансформеру с помощью класса типов

```
class MonadTrans t where
  lift :: Monad m => m a -> t m a
```

todo Реализуйте `instance MonadTrans (StateT s)`

todo Сконструируйте в терминах трансформеров следующее вычисление

```
computation :: Int -> (Int, String)
computation = \e -> (e + 2, show e ++ show (e + 1))
```

```
⚡ computation = runIdentity . runWriterT . runReaderT do
  x <- ask; lift (tell $ show x)
  y <- asks (+ 1); lift (tell $ show y)
  pure (y + 1)
```

Неявный lifting (библиотека mtl)

- ▶ Перегрузим функции стандартных монад по результирующей монаде

```
class MonadReader e m | m -> e where
  ask :: m e
```

- ▶ Для каждого трансформера определим, как в нем должна работать операция

```
instance MonadReader e m => MonadReader e (StateT s m) where
  ask = lift ask
```

- ▶ Можем вообще забыть тип трансформера и требовать от него конкретные свойства

```
computation :: (MonadReader String m, MonadState Int m) => m a
```

Анализ трансформеров

Monad Transformers and Modular Interpreters*

Sheng Liang

Paul Hudak

Mark Jones[†]

- + Позволяют инкрементально и модульно реализовывать функциональность eDSLs
- + Можно делать код полиморфным по стеку трансформеров, требуя интерфейсы
- Сложны в программировании и изучении (no chances mainstream)
- Башни существенно замедляют программу, особенно если стек абстрагирован
- Требуют существенного количества языковых фич для реализации и использования
- FunDeps ограничивают каждый слой в стеке одним экземпляром
- Требуют для каждого трансформера реализовать каждый интерфейс
- Порядок трансформеров играет роль, её можно понять только из реализации трансформера, множество упорядочений бесполезны

Содержание

1. Recall: monads
2. Love. Death. Transformers
3. Материалы

Что посмотреть в транспорте

- ▶ (post) The monads of Haskell — Mark Karpov
- ▶ [Haskell'23] The Evolution of Effects
- ▶ Alexis King - “Effects for Less” ZuriHac 2020
- ▶ “Building Haskell Programs with Fused Effects” by Patrick Thomson

- ▶ Oleg Kiselyev Blog — Having an Effect
- ▶ Liang, S., Hudak, P. and Jones, M., 1995, January. Monad transformers and modular interpreters. In Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages (pp. 333-343).
- ▶ Schrijvers, T., Piróg, M., Wu, N. and Jaskelioff, M., 2019, August. Monad transformers and modular algebraic effects: what binds them together. In Proceedings of the 12th ACM SIGPLAN International Symposium on Haskell (pp. 98-113).
- ▶ OPLSS 2018 – Andrej Bauer – Algebraic Effects and Handlers